Відокремлений структурний підрозділ

«Фаховий коледж інформаційних технологій»

Національного університету «Львівська політехніка»



лабораторної роботи

з дисципліни « WEB-програмування »

Виконав:

Студент групи КН-43

Гарбіч Богдан Назарович

Перевірив:

викладач

Роман Скалоцький

Львів 2026

ЗВІТ

з лабораторної роботи №5

Тема: Проєктування системи класів у JavaScript на прикладі навчального вебпроєкту «Акустична варта»

1. Мета роботи

Навчитися переходити від набору окремих об’єктів і функцій (процедурного підходу) до спроєктованої системи взаємопов’язаних класів у JavaScript (ES6). Виділити сутності предметної області «Акустична варта», визначити їх стан і поведінку, розподілити відповідальності, побудувати композицію та інтегрувати об’єктну модель у наявний вебінтерфейс з використанням ES6-модулів (`import`/`export`).

2. Постановка задачі та обґрунтування рефакторингу

У попередніх лабораторних роботах дані про датчики шуму зберігалися у звичайних об'єктах (plain objects), а їх обробка здійснювалася процедурними функціями. В рамках цієї лабораторної роботи було виконано рефакторинг JavaScript-частини:

1. Перетворено дані сенсорів на екземпляри класів (`NoiseSensor`, `NoiseMeasurement`).
2. Організовано керування колекцією через сервіс `MonitoringSystem`.
3. Централізовано бізнес-правила класифікації шуму (межі 55 dB та 70 dB) в єдиній конфігурації `NOISE_LIMITS`.
4. Розділено код на ES6-модулі (`models/`, `services/`, `app.js`).
5. Забезпечено відображення даних екземплярів у DOM-інтерфейсі без змішування бізнес-логіки з представленням сторінки.

3. Архітектура та відповідальності класів

3.1. Таблиця відповідальностей

Клас	Що моделює	Основні властивості (Стан)	Основні методи (Поведінка)	Взаємодія
`NoiseMeasurement`	Одне вимірювання	`value`, `measuredAt`	`isValid()`, `toString()`	Входить до складників `NoiseSensor`

	рівня шуму			
NoiseSensor	Фізичний датчик у точці міста	id, district, location, noiseLevel, measurements	getStatus(), updateNoiseLevel(), addMeasurement(), static isValidNoiseLevel(), static fromObject()	Зберігається в MonitoringSystem, містить NoiseMeasurement
MonitoringSystem	Сервіс управління колекцією	sensors	addSensor(), findSensorById(), getSensorsByDistrict(), getCriticalSensors(), getAverageNoiseLevel()	Керує масивом екземплярів NoiseSensor

3.2. Обґрунтування композиції та бізнес-правил

- **Композиція 1 (`MonitoringSystem` $\rightarrow$ `sensors`):** Клас `MonitoringSystem` містить масив об'єктів `NoiseSensor`, а не лише їхні ID чи числові значення.
- **Композиція 2 (`NoiseSensor` $\rightarrow$ `measurements`):** Датчик зберігає масив об'єктів `NoiseMeasurement`, що моделює історію замірів.
- **Централізація меж:** Межі шуму винесені в константу `NOISE_LIMITS` (`normalMax: 55`, `warningMax: 70`). Метод `getStatus()` порівнює значення з цією конфігурацією, що усуває дублювання чисел 55 та 70 в коді.

4. Реалізація коду (Структура ES6-модулів)

4.1. `js/models/NoiseMeasurement.js`

```
export class NoiseMeasurement {
    constructor(value, measuredAt = new Date()) {
        this.value = value;
        this.measuredAt = measuredAt;
    }
```

```
    isValid() {
        return Number.isFinite(this.value) && this.value >= 0 &&
this.value <= 200;
    }

    toString() {
        return `${this.value} dB
(${this.measuredAt.toLocaleTimeString()})`;
    }
}
```

4.2. js/models/NoiseSensor.js

```
export const NOISE_LIMITS = Object.freeze({
    normalMax: 55,
    warningMax: 70,
});

export class NoiseSensor {
    constructor(id, district, location, noiseLevel) {
        this.id = id;
        this.district = district;
        this.location = location;
        this.noiseLevel = noiseLevel;
        this.measurements = [];
    }

    getStatus() {
        if (this.noiseLevel <= NOISE_LIMITS.normalMax) {
            return "normal";
        } else if (this.noiseLevel > NOISE_LIMITS.normalMax &&
this.noiseLevel <= NOISE_LIMITS.warningMax) {
            return "warning";
        } else {
            return "critical";
        }
    }

    updateNoiseLevel(value) {
        if (!NoiseSensor.isValidNoiseLevel(value)) {
            throw new Error(`Недопустиме значення шуму: ${value}`);
```

```
        }
        this.noiseLevel = value;
    }

    addMeasurement(measurement) {
        if (!measurement.isValid()) {
            throw new Error("Некоректне вимірювання");
        }
        this.measurements.push(measurement);
        this.noiseLevel = measurement.value;
    }

    static isValidNoiseLevel(value) {
        return Number.isFinite(value) && value >= 0 && value <= 200;
    }

    static fromObject(data) {
        return new NoiseSensor(
            data.id,
            data.district,
            data.location || "Не вказано",
            data.noiseLevel || 0
        );
    }
}
```

4.3. js/services/MonitoringSystem.js

```
import { NoiseSensor } from '../models/NoiseSensor.js';

export class MonitoringSystem {
    constructor() {
        this.sensors = [];
    }

    addSensor(sensor) {
        if (!(sensor instanceof NoiseSensor)) {
            throw new Error("Очікується екземпляр NoiseSensor");
        }
        if (this.sensors.some(item => item.id === sensor.id)) {
            throw new Error(`Датчик ${sensor.id} вже існує`);
```

```
        }
        this.sensors.push(sensor);
    }

    findSensorById(id) {
        return this.sensors.find(sensor => sensor.id === id) ||
null;
    }

    getSensorsByDistrict(district) {
        return this.sensors.filter(sensor => sensor.district ===
district);
    }

    getCriticalSensors() {
        return this.sensors.filter(sensor => sensor.getStatus() ===
"critical");
    }

    getAverageNoiseLevel() {
        if (this.sensors.length === 0) return 0;
        const sum = this.sensors.reduce((acc, sensor) => acc +
sensor.noiseLevel, 0);
        return Number((sum / this.sensors.length).toFixed(2));
    }
}
```

4.4. js/app.js

```
import { NoiseSensor } from './models/NoiseSensor.js';
import { MonitoringSystem } from './services/MonitoringSystem.js';

const sensorsData = [
    { id: "NS-01", district: "Галицький", location: "просп.
Свободи", noiseLevel: 58 },
    { id: "NS-02", district: "Франківський", location: "вул.
Наукова", noiseLevel: 74 },
    { id: "NS-03", district: "Шевченківський", location: "просп.
Чорновола", noiseLevel: 83 },
];
```

```
const monitoringSystem = new MonitoringSystem();
sensorsData.forEach(data =>
monitoringSystem.addSensor(NoiseSensor.fromObject(data)));

function renderSensorCard(sensor) {
    const status = sensor.getStatus();
    const card = document.createElement('div');
    card.className = `sensor-card ${status}`;
    card.innerHTML = `
        <h3>ID: ${sensor.id}</h3>
        <p><strong>Район:</strong> ${sensor.district}</p>
        <p><strong>Локація:</strong> ${sensor.location}</p>
        <p><strong>Рівень шуму:</strong> ${sensor.noiseLevel} dB</p>
        <p><strong>Статус:</strong> ${status.toUpperCase()}</p>
    `;
    return card;
}

document.addEventListener('DOMContentLoaded', () => {
    const container = document.getElementById('sensors-container')
|| document.body;
    monitoringSystem.sensors.forEach(sensor =>
container.appendChild(renderSensorCard(sensor)));

    // Консольні перевірки за вимогами методички
    const sA = monitoringSystem.findSensorById("NS-01");
    const sB = monitoringSystem.findSensorById("NS-02");
    console.log("getStatus однаковий у прототипі:", sA.getStatus ===
sB.getStatus);
    console.log("noiseLevel є власною властивістю:",
Object.hasOwn(sA, "noiseLevel"));
    console.log("getStatus є власною властивістю:",
Object.hasOwn(sA, "getStatus"));
    console.log("Static перевірка:",
NoiseSensor.isValidNoiseLevel(65));
});
```

5. Результати виконання та перевірки

1. **Запуск через локальний сервер:** Запуск виконано за допомогою локального HTTP-сервера (`python -m http.server 8000` / `Live Server`), що забезпечило успішне завантаження ES6-модулів без помилок CORS-політики.
2. **Візуальний рендеринг:** Дані датчиків відображаються на сторінці через екземпляри `NoiseSensor`. Значення 58 dB отримало статус `WARNING` ($55 < 58 \le 70$), а 74 dB та 83 dB — статус `CRITICAL` (>70). Середній рівень шуму обчислено автоматично: 71.67 dB.
3. **Результати консольних тестувань:**
    a. `sA.getStatus === sB.getStatus` $\rightarrow$ `true` (підтверджує винесення методу у `prototype`).
    b. `Object.hasOwn(sA, "noiseLevel")` $\rightarrow$ `true` (стан є власною властивістю екземпляра).
    c. `Object.hasOwn(sA, "getStatus")` $\rightarrow$ `false` (метод знаходить з прототипу).

6. Відповіді на контрольні запитання

1. **Чим клас відрізняється від конкретного екземпляра?**

Клас — це шаблон (креслення), який описує структуру та поведінку. Екземпляр — конкретний об'єкт, створений за допомогою `new`, який має власний стан.

2. **Чому `noiseLevel` є власною властивістю, а `getStatus()` ні?**

`noiseLevel` задається у `constructor` для кожного об'єкта індивідуально. Метод `getStatus()` оголошений у тілі класу та винесений у `NoiseSensor.prototype`, тому спільно використовується всіма екземплярами.

3. **Де фактично знаходиться метод класу?**

У об'єкті-прототипі відповідного класу (`NoiseSensor.prototype`).

4. **Коли операцію доцільно зробити `static`?**

Коли логіка пов'язана з сутністю, але не вимагає стану конкретного екземпляра (не використовує `this`). Наприклад, перевірка діапазону `NoiseSensor.isValidNoiseLevel(value)`.

5. **Чому `renderSensorCard` не є методом `NoiseSensor`?**

Створення HTML-елементів є логікою інтерфейсу користувача (UI), а не фізичною поведінкою датчика. Відокремлення запобігає змішуванню предметної області з DOM-кодом.

7. Висновки

У ході лабораторної роботи було виконано перехід від процедурного коду до об'єктно-орієнтованої архітектури у JavaScript. Створено класи `NoiseSensor`, `NoiseMeasurement` та сервіс `MonitoringSystem`, застосовано композицію об'єктів, централізовано бізнес-правила та розбито проєкт на модулі. Робота системи була повністю перевірена в браузері та консолі розробника.